

Aggregates en Domain-Driven Design

En Domain-Driven Design (DDD), un *Aggregate* es un conjunto de objetos del dominio que se agrupan porque deben mantenerse consistentes entre sí. El aggregate define un límite dentro del modelo de objetos, indicando qué entidades forman una unidad lógica de negocio. Su propósito es evitar que cualquier parte del sistema pueda modificar libremente cualquier entidad, reduciendo el acoplamiento y protegiendo las reglas importantes del dominio.

Cada aggregate posee una entidad principal denominada *Aggregate Root* o raíz del aggregate. Esta raíz representa el único punto de acceso válido desde el exterior hacia los objetos internos. Toda modificación relevante debe realizarse a través de ella, ya que es la encargada de validar reglas de negocio y garantizar la consistencia interna del aggregate.

Características principales de un Aggregate

- Posee una única raíz (*Aggregate Root*).
- Define límites claros de consistencia.
- Encapsula entidades y objetos relacionados.
- Controla las modificaciones internas.
- Representa una unidad lógica de persistencia.
- Reduce el acoplamiento entre partes del dominio.

Uno de los conceptos más importantes en DDD es la consistencia. Un aggregate existe para garantizar que ciertas reglas del dominio se cumplan siempre luego de cada operación. Estas reglas, llamadas invariantes, representan restricciones críticas del negocio.

Ejemplos de invariantes

- Un pedido no puede quedar sin items.
- Una colección no puede contener elementos duplicados.
- Una solicitud no puede aprobarse dos veces.
- Un curso publicado debe contener módulos válidos.

Por este motivo, los aggregates funcionan como límites de consistencia. Todo aquello que deba mantenerse consistente inmediatamente suele pertenecer al mismo aggregate.

Otro aspecto importante es el ciclo de vida de las entidades internas. Muchas veces, los objetos contenidos dentro de un aggregate dependen completamente de la raíz para existir. Esto implica que

nacen, se modifican y desaparecen junto con ella.

Ejemplo de ciclo de vida compartido

- Un `ItemPedido` no tiene sentido sin un `Pedido`.
- Una `Leccion` pertenece a un `Curso`.
- Un `ComentarioInterno` depende de su publicación.

En cambio, las entidades que poseen identidad y existencia independiente suelen modelarse como aggregates separados.

Las modificaciones sobre un aggregate deben realizarse mediante métodos definidos en la raíz. Esto evita exponer estructuras mutables directamente y permite centralizar la lógica de negocio.

Buenas prácticas al diseñar Aggregates

- Mantener aggregates pequeños y enfocados.
- Encapsular las colecciones internas.
- Evitar modificaciones directas sobre entidades internas.
- Colocar la lógica relevante dentro del dominio.
- Referenciar otros aggregates mediante identificadores.
- Separar aggregates que puedan evolucionar independientemente.

Desde el punto de vista de persistencia, un aggregate suele considerarse una unidad completa de almacenamiento y recuperación. Aunque inicialmente no se utilicen bases de datos, este concepto resulta importante para justificar el diseño del modelo. En una arquitectura completa de DDD, normalmente un repositorio recupera y persiste aggregates completos, ya que representan estructuras consistentes del dominio.

Persistencia y referencias entre Aggregates

DDD recomienda que los aggregates se relacionen mediante identificadores y no mediante referencias directas complejas.

Por ejemplo:

```
class Pedido {  
    private ClienteId clienteId;  
}
```

En lugar de:

```
class Pedido {  
    private Cliente cliente;  
}
```

Esto reduce el acoplamiento y evita dependencias innecesarias entre diferentes partes del dominio.

Organización en paquetes

Una estrategia frecuente consiste en organizar el proyecto con un paquete por aggregate:

```
dominio/  
├── curso/  
│   ├── Curso.java  
│   ├── Modulo.java  
│   └── Leccion.java  
├── inscripcion/  
│   ├── Inscripcion.java  
│   └── ProgresoAlumno.java  
└── alumno/  
    └── Alumno.java
```

Esta organización mejora la separación de responsabilidades y facilita identificar los límites de consistencia dentro del sistema.

Ejemplo

